Seylan Sensor: Product Browsing Kiosk App

How it works and how we would build it

1. The requirement

Seylan Bank wants customers using the Emojot Sensor on branch tablets to tap **"Browse the Product"**, look at product pages on seylan.lk, and then come back to the Sensor. Seylan's website team will not make any changes on their side.

In a normal browser this cannot be done reliably:

- Once the customer is on seylan.lk, our Sensor's code no longer runs, so we cannot tell whether they are still using the tablet.
- The customer can open several pages or new tabs, and a web page cannot close tabs it did not open or force a return to the Sensor.
- The next customer could find the previous customer's pages still open.

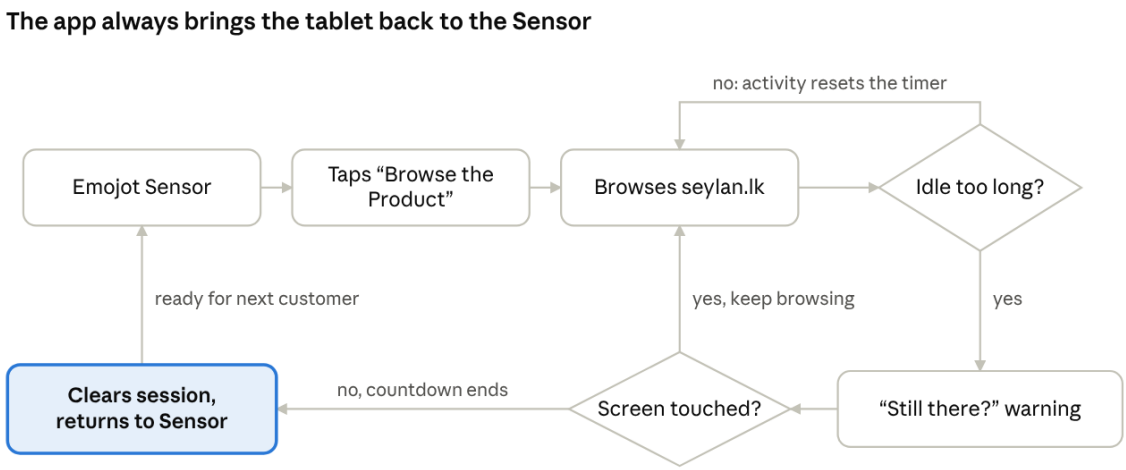
2. The solution in brief

We build a small Android app (React Native with Expo) and install it on the bank-owned tablets. The app becomes the only thing the customer uses. Inside it:

- The existing Emojot Sensor loads full screen, exactly as it does today.
- "Browse the Product" opens seylan.lk inside the app, in a controlled in-app browser (a WebView), with a fixed "Back to Sensor" bar.
- Because the app owns that browser, it can see every page change and every touch, keep everything in one view, and close it whenever it needs to.
- If the customer stops using the tablet, the app warns them, then clears their session and returns to the Sensor.

Nothing changes on Seylan's website or in the Emojot Sensor.

3. How it works, step by step



Customer journey: the app always brings the tablet back to the Sensor

1. **The tablet starts on the Sensor.** The app opens straight into the Emojot Sensor, full screen. The customer answers it as usual and responses go to Emojot as normal.
2. **The customer taps "Browse the Product".** The Sensor tries to open seylan.lk. The app catches that link before it loads and opens it in a separate browse view instead.
3. **The customer browses Seylan's site.** They can move between product pages freely. Any link that would open a new tab opens in the same view, so there is only ever one page open.
4. **The app watches for activity.** Every touch, scroll or key press on any Seylan page is reported to the app, and every new page load also counts as activity. Each one resets an idle timer.
5. **If the customer stops, the app warns them.** When the idle time runs out (for example 90 seconds), a "Still there?" message appears with a short countdown (for example 15 seconds).
6. **Touching the screen continues browsing.** The timer restarts and the customer carries on.
7. **If nobody touches it, the app resets.** The browse view closes, the customer's cookies, cache and anything typed into forms are cleared, and the Sensor is shown again, ready for the next customer.
8. **The customer can also leave any time.** The fixed "Back to Sensor" bar takes the same clear-and-return path immediately.

4. How each part happens

This section explains the mechanism behind each step. Code is shortened to the essential lines.

4.1 Loading the Sensor

The app shows the Sensor in a WebView, the in-app browser component (react-native-webview). It loads the same Sensor URL used today, with the sensor or device ID in the link, so nothing changes on the Emojot side.

4.2 Catching "Browse the Product"

Before any page loads, the WebView asks the app whether to allow it (onShouldStartLoadWithRequest). When the Sensor tries to go to seylan.lk, the app says no to the Sensor view and opens that address in the browse view instead. The Sensor stays loaded underneath.

```
<WebView
  source={{ uri: SENSOR_URL }}
  setSupportMultipleWindows={false}
  onShouldStartLoadWithRequest={req => {
    if (req.url.includes('seylan.lk')) {
      openBrowse(req.url);          // show the Seylan browse view
      return false;                 // keep the Sensor where it is
    }
    return true;
  }}
/>
```

4.3 Keeping everything in one view (no tabs)

A WebView has no tabs. With setSupportMultipleWindows set to false, any link marked "open in new tab" and any window.open call simply loads in the same view. The customer can never end up with pages the app does not control.

4.4 Detecting activity on Seylan's pages

We cannot change Seylan's website, but the app can add a small script to every page it loads (injectedJavaScript). The script runs inside each Seylan page and sends a message to the app whenever the customer touches, scrolls, clicks or types. The app also treats every page load as activity.

```
const ACTIVITY_LISTENER = `
  (function () {
    var send = function () { window.ReactNativeWebView.postMessage('activity'); };
    ['touchstart', 'scroll', 'click', 'keydown'].forEach(function (e) {
      window.addEventListener(e, send, { passive: true, capture: true });
    });
  })();
  true;
`;
```

4.5 The idle timer and warning

Each activity message restarts a timer in the app. When it runs out, the app shows the "Still there?" overlay with a countdown. A touch closes the overlay and restarts the timer; if the countdown ends, the app returns to the Sensor. Both times are settings.

```
const IDLE_MS = 90000, WARN_MS = 15000;     // example values

const resetIdleTimer = () => {
  clearTimeout(idle.current);
  idle.current = setTimeout(showWarning, IDLE_MS);
};

const showWarning = () => {
  setWarning(true);
  idle.current = setTimeout(returnToSensor, WARN_MS);
};
```

4.6 Returning to the Sensor and clearing the session

The browse view runs in incognito mode, so nothing it stores outlives it. To return, the app closes the browse view and gives the next one a new key, which makes React Native create a completely fresh browser. The previous customer's cookies, cache and form entries are gone.

```
<WebView
  key={sessionKey}                      // new key = brand-new browser
  source={{ uri: browseUrl }}
  incognito                             // nothing kept between customers
  setSupportMultipleWindows={false}     // new-tab links stay here
  injectedJavaScript={ACTIVITY_LISTENER}
  onMessage={resetIdleTimer}
  onLoadStart={resetIdleTimer}
  onShouldStartLoadWithRequest={req => isAllowed(req.url)}
/>

const returnToSensor = () => {
  setWarning(false);
  setBrowsing(false);                   // close the Seylan view
  setSessionKey(k => k + 1);            // next customer starts fresh
};
```

4.7 Keeping customers on Seylan's site

The browse view only loads addresses on an allowlist (seylan.lk and the Sensor's domain). Other websites, phone and email links, and links that would open other apps are blocked.

```
const ALLOWED = ['seylan.lk', 'emojot.com'];
const isAllowed = url =>
  ALLOWED.some(d => new URL(url).hostname.endsWith(d));
```

4.8 Locking the tablet

- **Screen and display:** the app keeps the screen on (expo-keep-awake), fixes the orientation, and hides Android's navigation bar (expo-navigation-bar).
- **Full kiosk lock:** so customers cannot leave the app, the bank's device management (MDM) runs the tablet in single-app mode. If the bank has no MDM, Android screen pinning or a small native lock-task plugin does the same job.

4.9 Connection to Emojot

The Sensor is unchanged and keeps sending responses to Emojot. Two optional additions make the app easier to run across many tablets:

- **Settings from Emojot:** idle time, warning length and allowed domains read from Emojot, so they change without reinstalling the app.
- **Usage events to Emojot:** "Browse the Product" tapped, returned by timeout and returned by button, for reporting.

5. React Native with Expo Or Flutter

Both frameworks can build this app. React Native with Expo is the better fit because the app is mostly an in-app browser, it must be easy to update on tablets already in branches, and the team already works in JavaScript.

Factor	React Native with Expo	Flutter
In-app browser (the core of this app)	react-native-webview is mature and covers everything needed: catching links, adding the activity script, keeping new tabs in one view, incognito	The official webview_flutter is more limited; likely needs the third-party flutter_inappwebview for new-window handling and session clearing
Updating tablets in the field	EAS Update pushes fixes over the air (for example a new idle time or allowed domain) without reinstalling	No built-in equivalent; needs a reinstall or a third-party service (Shorebird)
Building the install file	eas build in the cloud, no Android Studio setup needed	Local Android toolchain or a CI service
Kiosk lock	Bank MDM or a small native plugin	Same
Performance	Plenty for an app that is mainly web views	Slightly faster custom UI, which this app barely has

Recommendation: React Native with Expo. Flutter would only be the better choice if the team already had strong Flutter experience.

6. Technical build

Part	Choice	Purpose
App framework	React Native with Expo	One JavaScript codebase, fast builds, quick updates
In-app browser	react-native-webview	Shows the Sensor and Seylan's pages, and lets the app watch and control them
Screen settings	expo-keep-awake, expo-navigation-bar	Screen always on, navigation bar hidden
Build	EAS Build (Expo's build service)	Produces the Android install file (APK)
Kiosk lock	Bank MDM single-app mode, or lock-task plugin	Stops customers leaving the app

App structure

- **App.tsx:** holds the Sensor view and the browse view, and switches between them.
- **BrowseView.tsx:** the Seylan WebView, "Back to Sensor" bar, activity script and allowlist.
- **IdleWarning.tsx:** the "Still there?" overlay and countdown.
- **config.ts:** Sensor URL, allowed domains, idle and warning times.

Build and install

```
npx create-expo-app seylan-kiosk
npx expo install react-native-webview expo-keep-awake expo-navigation-bar
eas build -p android --profile preview      # profile set to buildType "apk"
```

The resulting APK is installed directly on the tablet or pushed through the bank's MDM. No Play Store listing is needed.

7. Feasibility and risks

Feasible. Every part above uses standard, well-supported components. The points below need checking on the bank's actual tablet.

Risk	Why it matters	How we handle it
Seylan's site in a WebView	Some sites behave differently inside in-app browsers	Use a standard browser user agent; test the key product pages
PDF brochures	Android WebView cannot show PDFs	Block PDF links or open them through a viewer page
Tablet model and Android version	Old WebView versions may not render the site well	Confirm the model and keep its WebView updated
Kiosk lock	Without it, customers can leave the app	Bank MDM single-app mode; otherwise screen pinning or lock-task plugin
Privacy on a shared tablet	One customer's details must not reach the next	Incognito browse view, fully replaced on every return

8. Scope

In scope

- Android app showing the existing Emojot Sensor full screen
- "Browse the Product" opening seylan.lk inside the app with a "Back to Sensor" bar
- Browsing across product pages with new-tab links kept in one view
- Inactivity detection, "Still there?" warning and automatic return to the Sensor
- Clearing each customer's session on return
- Domain allowlist (seylan.lk and the Sensor domain)
- Tablet settings: screen on, fixed orientation, hidden navigation bar
- Android install file (APK) for direct install or the bank's MDM

Out of scope

- Any change to Seylan Bank's website
- Any change to the Emojot Sensor
- iOS / iPad
- Setting up the bank's device management (we use their existing setup)
- Play Store publishing

Needed from Seylan Bank

- Tablet model and Android version (ideally one tablet for testing if we go with reactnative and expo it need an android version above 7)
- Whether their device management supports single-app kiosk mode
- The Sensor link(s) and the seylan.lk page "Browse the Product" should open
- Preferred idle time before the warning